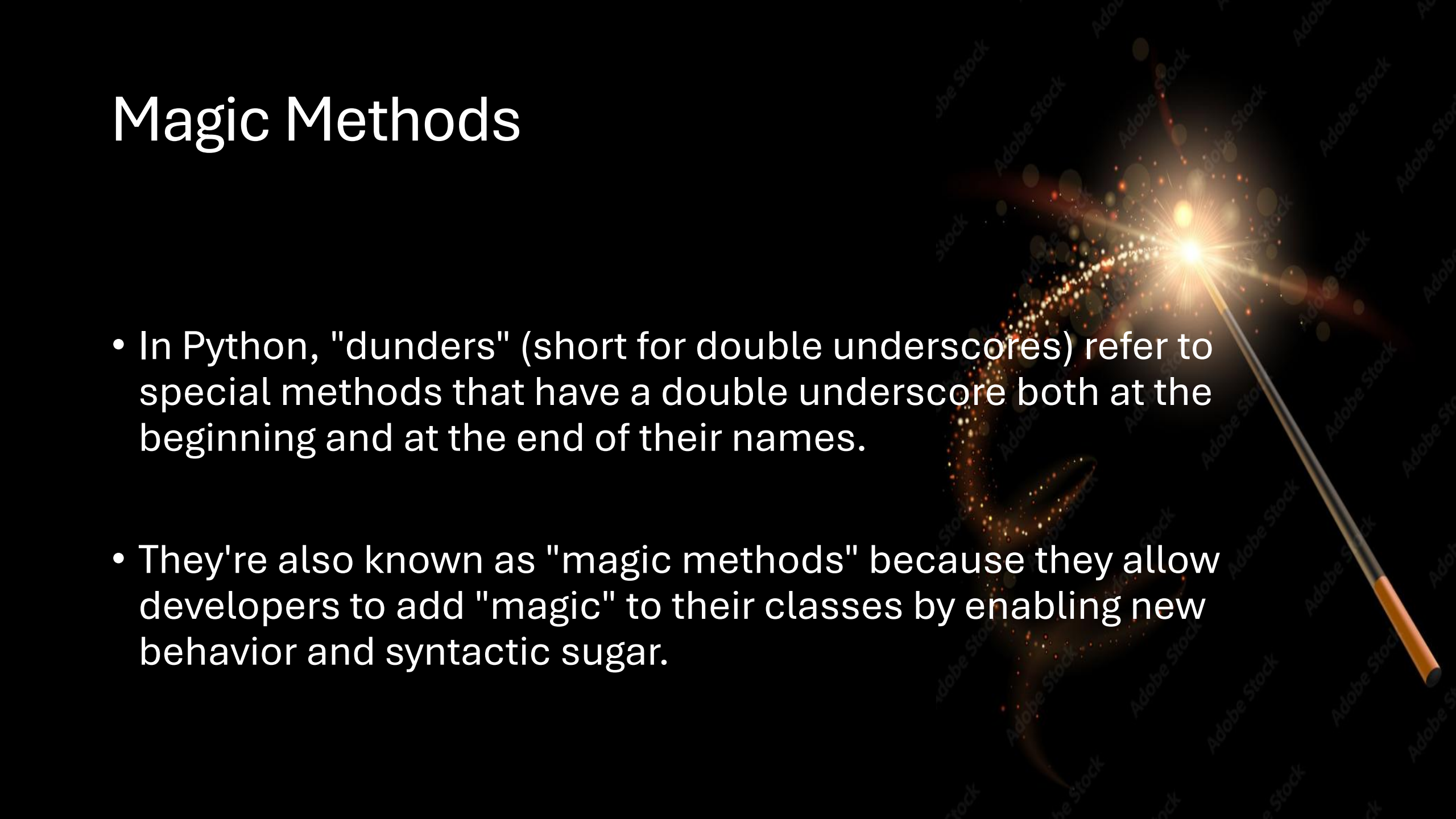

__dunders__

Magic Methods



- In Python, "dunders" (short for double underscores) refer to special methods that have a double underscore both at the beginning and at the end of their names.

- They're also known as "magic methods" because they allow developers to add "magic" to their classes by enabling new behavior and syntactic sugar.

Operator Overloading

- Dunders like __add__, __sub__, __mul__, etc., allow you to define how operators work with your custom objects.

- For example, if you have a custom vector class, you can use these methods to define how vector addition should work.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2  # This calls v1.__add__(v2)
```

Object Initialization and Representation

- Methods like __init__, __repr__, and __str__ are crucial for object creation and debugging.

```
class Person:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f'Person({self.name})'

    def __str__(self):
        return f'My name is {self.name}'

p = Person('Alice')
print(p)  # Calls p.__str__()
```

Attribute Access Control

- Methods like __getattr__, __setattr__, and __delattr__ let you control access to attributes.

```
class SafeDict:
    def __init__(self, **kwargs):
        self.__dict__.update(kwargs)

    def __getattr__(self, name):
        return f"{name} not found!"

sd = SafeDict(a=1, b=2)
print(sd.a)  # Outputs: 1
print(sd.c)  # Outputs: c not found!
```

Context Managers

- The __enter__ and __exit__ methods enable you to manage resources like files, ensuring they are properly closed after use.

```
class ManagedFile:
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        self.file = open(self.name, 'w')
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()

with ManagedFile('test.txt') as f:
    f.write('hello, world!')
```

Custom Iterators

- Methods like __iter__ and __next__ enable objects to be iterable.

```
class Counter:
    def __init__(self):
        self.count = 0

    def __iter__(self):
        return self

    def __next__(self):
        self.count += 1
        return self.count

counter = Counter()
print(next(counter))  # Outputs: 1
print(next(counter))  # Outputs: 2
```

Core & Attributes	Containers & Iteration	Operators & Class Mechanics
__new__ – create object __init__ – initialize __del__ – cleanup __str__ – user string __repr__ – dev string __bytes__ – to bytes __format__ – formatting __getattr__ – missing attr __getattribute__ – all attr access __setattr__ – assign attr __delattr__ – delete attr __dir__ – customize dir()	__len__ – length __getitem__ – indexing __setitem__ – assign index __delitem__ – delete index __iter__ – iterator start __next__ – iterator step __contains__ – membership __call__ – callable object __enter__ – context enter __exit__ – context exit	__add__, __sub__, __mul__, __truediv__ __radd__, __rsub__, etc. __iadd__, __isub__, etc. __neg__, __pos__, __abs__ __eq__, __ne__ __lt__, __le__, __gt__, __ge__ __hash__ – hashing __bool__ – truthiness __init_subclass__ – subclass hook __instancecheck__ – custom isinstance __subclasscheck__ – custom issubclass __set_name__ – descriptor binding __and__, __or__, __xor__, __lshift__, __rshift__

Summary

- Dunders are the stars of Python, making it incredibly flexible and expressive.
- Dunders can make user-defined objects behave like Python’s built-in types.
- Embracing them can elevate your programming prowess to new heights.

Dunder Tutorial

https://www.geeksforgeeks.org/dunder-magic-methods-python/